

Homework 04 – Theme Park Fun!

Authors: Sabina, Chloe, Helen, Ricky, Sang Yoon

Topics: polymorphism, dynamic binding, interfaces, comparable

Problem Description

A new amusement park has opened around Georgia Tech called “4 Flags”. However, with all the different attractions in the park to manage, you decide to use your skills in object-oriented programming to make it easier to maintain the amusement park. In this homework, you will use your knowledge of polymorphism to help with the management of many different attractions.

Solution Description

You will need to complete and turn in one interface and four classes: `Admittable.java`, `Group.java`, `Attraction.java`, `RollerCoaster.java`, and `TripGuide.java`.

Notes:

1. All variables should be inaccessible from other classes and require an instance to be accessed through, unless specified otherwise.
2. All methods should be accessible from everywhere and must require an instance to be accessed through, unless specified otherwise.
3. Use constructor chaining and reuse methods whenever possible! Ensure that your constructor chaining helps you maximize code reuse!
4. Reuse code when possible and helper methods are optional.
5. Make sure to add Javadoc comments to your methods and classes!

`Admittable.java`

This file represents an interface that enables implementing classes to define the behavior of admittance.

Methods:

- `admit`
 - Takes in a `String[]` corresponding to the name(s) of the visitor(s) to be admitted.
 - Does not return anything.
 - This method should not be implemented in `Admittable`.

`Group.java`

This class represents a group of visitors visiting an attraction as a group.

Variables:

- `String[] people` – represents a group of visitor(s) that are visiting an attraction together
 - This field should be **immutable**.
 - **Note:** Assume the individual elements of the array will be valid (i.e., non-null, non-blank).

Constructor:

- A constructor that takes in a `String[]` of people.
 - If the reference passed in is `null`, a new array should be created with length 0.
 - If the reference passed in is non-null, a **deep copy** of the input array should be made.

Methods:

- `size`
 - Returns an `int` representing the size of the group (i.e., the number of people in the group).
- `toString`
 - This method should properly override `Object`'s `toString` method.
 - Returns the names of the visitor(s) in the group with forward-slash (i.e., `"/"`) in between names.
 - For example, a group of three people should return the following string (without quotation marks, quotation marks, and trailing/leading whitespace):

```
"<name1>/<name2>/<name3>"
```
- No getters and setters.

Attraction.java

This class represents an attraction and implements the `Admittable` interface. Attractions can be compared to other attractions based on their rating and admission fee and should follow the contract set by the `Comparable` interface using generics.

Variables:

- `String name` – the name of this attraction
 - This field should be **immutable**.
 - In the case that we try to instantiate this attraction with a null value or blank string, the name should default to "No name".
 - **Hint:** Remember that the `String` class provides a method to test whether a string is blank (i.e., composed entirely of whitespace characters).
- `long sumRatings` – the sum of all the ratings this attraction has received, initially 0
- `int numRatings` – the total number of ratings this attraction has received, initially 0
- `double admissionFee` – the fee for going into this attraction
 - This field should be **immutable**.
 - In the case that we try to instantiate this attraction with a negative admission fee, the admission fee should be set to 0.0.
- `Group[] visitors` – an array of visitors
 - This array should be initialized as an empty array of length 5.
 - Groups in the `visitors` array should be placed starting at index 0 and must **always** be kept contiguous (i.e., non-null elements should not be separated by null element(s)).

Constructors:

- A constructor that takes in `name` and `admissionFee`.
 - These values should conform to the specifications stated above.
- A constructor that takes in `name` and sets `admissionFee` to 5.25.

Methods:

- `admit`
 - Overrides `admit` from the `Admittable` interface.
 - Takes in a `String[]` representing the name(s) of the new visitor(s) to be added and does not return anything.
 - **Note:** Assume the input will be non-null and the names in the array will be valid (i.e., non-null, non-blank).
 - Each visitor should first be added to a group before being added to `visitors`.
 - Groups have a maximum capacity of 5. If more than five new visitors are admitted, groups of five must be created until there are no more than five visitors remaining. The remaining visitor(s) should be placed in a group together.
 - Groups should be added starting at the first empty index in `visitors`.
 - If `visitors` is full, the array must be resized to double the current length. Make sure the existing visitor(s) are copied to the same indices in the new resized array and that the new groups are also added to the resized array.
 - **Note:** You may need to resize `visitors` more than once within a single invocation of `admit`.
 - **Hint:** Helper methods would be great to use here! For instance, a helper method could be created to help resize the `visitors` array. Another helper method could be written to create a copy of a range of elements within an array. Making helper methods will make it easier to test your code.
- `rateAndExit`
 - This method removes a group of visitors from the attraction and updates the attraction's rating based on their response.
 - Takes in an `int` corresponding to the index of a group in `visitors` and an `int` corresponding to the rating the group will give the attraction.
 - If the index passed in is invalid or no group exists at that index, do not update the rating, return -1, and print the following string on its own line (without quotation marks):

```
"Could not update rating. Index invalid."
```
 - Returns an `int` representing the number of visitor(s) removed.
 - Updates both `sumRatings` and `numRatings` based on the rating given by the group that left.
 - The rating must be an `int` in the range [1, 10]. If a number less than 1 is given, then a rating of 1 should be used. If a number greater than 10 is given, then a rating of 10 should be used.
 - **Note:** One group of `visitors`, regardless of how big the group is, can only yield one rating.
 - Remove the group from the attraction by removing the group from the array and shifting the remaining groups in the array up (i.e., towards index 0) so that all non-null values in the array are still contiguous (i.e., next to each other) starting at index 0.

THIS GRADED ASSESSMENT IS NOT FOR DISTRIBUTION.
ANY DUPLICATION OUTSIDE OF GEORGIA TECH'S LMS IS UNAUTHORIZED.

- `averageRating`
 - This method should return a `double` with the average rating rounded to two decimal places. Remember that `sumRatings` and `numRatings` are integer types, but the average rating should include decimal points.
 - **Hint:** `Math.round` might be helpful here! See the Math API to read about how this method works.
 - If the number of ratings is zero, then an average rating of 0 should be returned.
- `printVisitors`
 - This method should print information about the attraction and the visitor groups currently in the attraction.
 - The string representation of this attraction should be printed on its own line, followed by all the groups in `visitors`.
 - The groups in the `visitors` array in the following format (without quotation marks and angle brackets, each group followed by a single newline character):

```
Group 1: <Group1>
Group 2: <Group2>
. . .
Group N: <Group5>
"
```
 - **Note:** Null values in the `visitors` array should not be printed. `N` represents the number of non-null elements in `visitors`.
- `toString`
 - Overrides `toString` from `Object`.
 - Returns the following string with all floating-point values displayed to two decimal places with rounding in the following format (without quotation marks, angle brackets, and trailing/leading whitespace):

```
"<name>/<averageRating>/<admissionFee>"
```
- `compareTo`
 - Overrides the method in the `Comparable` interface with the same name.
 - Attractions should be compared first by average rating, followed by admission fee. Admission fee should only be compared if the ratings are the same.
 - If the **input** attraction is null, return a **negative** value.
 - If **this** attraction and the input attraction have the same average rating and admission fee, the result should be **0**.
 - If **this** attraction has a higher average rating than the input attraction, then a **positive** value should be returned. Otherwise, a **negative** value should be returned (if the attractions do not have the same average rating).
 - If the ratings are the same and this attraction has a higher admission fee than the input attraction, then a **positive** value should be returned. Otherwise, a **negative** value should be returned (if the attractions do not have the same admission fee).
- No getters and setters.

RollerCoaster.java

This class represents a roller coaster and is a subclass of `Attraction`. Roller coasters have a maximum number of visitors that are allowed to enter.

Variables:

- `int maxCapacity` – the maximum number of visitors this roller coaster can hold
 - This value cannot be less than 25. If we try to instantiate a roller coaster with a `maxCapacity` less than 25, it should default to a value of 25. Once instantiated, this value should **not** be changed.
- `int occupancy` – the current number of people on this roller coaster, initially 0

Constructors:

- A constructor that takes in `name`, `admissionFee`, and `maxCapacity`.
 - These values should conform to the specifications stated above.
- A constructor that takes in the `name` and sets `admissionFee` to 5.25 and `maxCapacity` to 25.

Methods:

- `admit`
 - Overrides `admit` from `Attraction`.
 - Takes in a `String[]` representing the name(s) of the new visitor(s) to be added and does not return anything.
 - **Note:** Assume the input will be non-null and the names in the array will be valid (i.e., non-null, non-blank).
 - This method has the same requirements as defined in the `admit` method of `Attraction`, with a few additional requirements.
 - **Hint:** Think about how you can reuse code to minimize the code written in this method!
 - `occupancy` should increase by the number of visitor(s) added to `visitors`.
 - If the visitor(s) to be added would cause this roller coaster to exceed its `maxCapacity`, reject all the visitor(s) and print the following string (without quotation marks):

```
"RollerCoaster has reached maximum capacity. Please visit another time!"
```
- `rateAndExit`
 - Overrides the `rateAndExit` method from `Attraction`.
 - Takes in an `int` corresponding to the index of a group in the `visitors` array and an `int` corresponding to the rating the group will give the attraction.
 - Returns an `int` representing the number of visitors removed.
 - This method has the same requirements as defined in the `rateAndExit` method of `Attraction`, with an additional requirement.
 - **Hint:** Think about how you can reuse code to minimize the code written in this method!
 - `occupancy` should decrease by the number of visitor(s) leaving this attraction.

THIS GRADED ASSESSMENT IS NOT FOR DISTRIBUTION.
ANY DUPLICATION OUTSIDE OF GEORGIA TECH'S LMS IS UNAUTHORIZED.

- `percentOccupancy`
 - This method should return the percentage of occupancy, calculated by using the values of `occupancy` and `maxCapacity`.
 - Returns a `double` rounded to two decimal places.
 - **Hint:** `Math.round` might be helpful here! See the Math API to read about how this method works.
 - For example, return the `double 12.34` for the calculated value 12.341%.
- `toString`
 - Overrides `toString` from `Attraction`.
 - Returns the following string with all floating-point values displayed to two decimal places with rounding in the following format (without quotation marks, angle brackets, and trailing/leading whitespace, single space following colon):

```
"RollerCoaster: <name>/<averageRating>/<admissionFee>/<percentOccupancy>%"
```

 - **Hint:** Think about what keyword minimizes the code written in this method.
- No getters and setters.

TripGuide.java

This class represents a trip guide that allows you to create and edit your trips. Use this as your driver class to test the functionality of all the code you've implemented!

Methods:

- `main`
 - Create an array of `Attractions` containing:
 - An `Attraction` with a name and admission fee of your choosing.
 - An `Attraction` with a different name and same admission fee as the first.
 - A `RollerCoaster` with a different name and same admission fee as the first.
 - An `Attraction` with a different admission fee.
 - A `RollerCoaster` with a different admission fee.
 - Any other `Attractions` or `RollerCoaster` of your choosing.
 - Create 3 arrays with different lists of people of varying size (e.g., 4 people, 5 people, 12 people, 25 people).
 - Use `admit` to admit these groups of people to all attractions.
 - Call `printVisitors` after each admittance to observe the changes happening!
 - Call `rateAndExit` on some attractions so that:
 - Two `Attractions` remove the same groups and thus have the same ratings.
 - Two `Attractions` remove different groups and thus have different ratings.
 - One `Attraction` stays fully occupied and thus its rating remains 0.
 - Call `toString` and `printVisitors` for each attraction in the array.
 - Use `compareTo` on the attractions. Consider scenarios where the rating is different, the rating is the same, but the admission fee is different, and both are the same.
 - Test any edge cases you think your code may run into!

Checkstyle

You must run Checkstyle on your submission (to learn more about Checkstyle, check out [cs1331-style-guide.pdf](#) under the Checkstyle Resources module on Canvas). **The Checkstyle cap for this assignment is 25 points.** This means there is a maximum point deduction of 25. If you don't have Checkstyle yet, download it from Canvas → Modules → Checkstyle Resources → [checkstyle-8.28.jar](#). Place it in the same folder as the files you want to run Checkstyle on. Run Checkstyle on your code like so:

```
$ java -jar checkstyle-8.28.jar YourFileName.java
Starting audit...
Audit done.
```

The message above means there were no Checkstyle errors. If you had any errors, they would show up above this message, and the number at the end would be the number of points we would take off (limited by the Checkstyle cap). In future assignments we will be increasing this cap, so get into the habit of fixing these style errors early!

Additionally, you must Javadoc your code.

Run the following to only check your Javadocs:

```
$ java -jar checkstyle-8.28.jar -j yourFileName.java
```

Run the following to check both Javadocs and Checkstyle:

```
$ java -jar checkstyle-8.28.jar -a yourFileName.java
```

For additional help with Checkstyle see the [CS 1331 Style Guide](#).

Turn-In Procedure

Submission

To submit, upload the files listed below to the corresponding assignment on Gradescope:

- `Admittable.java`
- `Group.java`
- `Attraction.java`
- `RollerCoaster.java`
- `TripGuide.java`

Make sure you see the message stating the assignment was submitted successfully. From this point, Gradescope will run a basic autograder on your submission as discussed in the next section. **Any autograder tests are provided as a courtesy to help “sanity check” your work and you may not see all the test cases used to grade your work.** You are responsible for thoroughly testing your submission on your own to ensure you have fulfilled the requirements of this assignment. If you have questions about the requirements given, reach out to a TA or Professor via the class forum for clarification.

You can submit as many times as you want before the deadline, so feel free to resubmit as you make substantial progress on the assignment. We will only grade your latest submission. **Be sure to submit every file each time you resubmit.**

Gradescope Autograder

If an autograder is enabled for this assignment, you may be able to see the results of a few basic test cases on your code. Typically, tests will correspond to a rubric item, and the score returned represents the performance of your code on those rubric items only. If you fail a test, you can look at the output to determine what went wrong and resubmit once you have fixed the issue. **We reserve the right to hide any or all test cases, so you should make sure to test your code thoroughly against the assignment's requirements.**

The Gradescope tests serve two main purposes:

- Prevent upload mistakes (e.g., non-compiling code)
- Provide basic formatting and usage validation

In other words, the test cases on Gradescope are by no means comprehensive. Be sure to thoroughly test your code by considering edge cases and writing your own test files. You also should avoid using Gradescope to compile, run, or Checkstyle your code; you can do that locally on your machine.

Other portions of your assignment can also be graded by a TA once the submission deadline has passed, so the output on Gradescope may not necessarily reflect your grade for the assignment.

Burden of Testing

You are responsible for thoroughly testing your submission against the written requirements to ensure you have fulfilled the requirements of this assignment.

Be **very careful** to note the way in which text output is formatted and spelled. Minor discrepancies could result in failed autograder cases.

If you have questions about the requirements given, reach out to a TA or Professor via the class forum for clarification.

Allowed Imports

To prevent trivialization of the assignment, you may not import any classes for this assignment.

Feature Restrictions

There are a few features and methods in Java that overly simplify the concepts we are trying to teach or break our autograder. For that reason, do not use any of the following in your final submission:

- `var` (the reserved keyword)
- `System.exit`
- `System.arraycopy`

Collaboration

Only discussion of the assignment at a conceptual high level is allowed. You can discuss course concepts and assignments broadly; that is, at a conceptual level to increase your understanding. If you find yourself dropping to a level where specific Java code is being discussed, that is going too far. Those discussions should be reserved for the instructor and TAs. To be clear, you should never exchange code related to an assignment with anyone other than the instructor and TAs.

The only code you may share are test cases written in a Driver class. If you choose to share your Driver class, they should be posted to the assignment discussion thread on the course discussion forum. We encourage you to write test cases and share them with your classmates, but we will not verify their correctness (i.e., use them at your own risk).

You **MAY NOT** use code generation tools to complete this assignment. This includes generative AI tools such as ChatGPT.

Important Notes (Don't Skip)

- Non-compiling files will receive a 0 for all associated rubric items.
- Do not submit `.class` files.
- Test your code in addition to the basic checks on Gradescope.
- Submit every file each time you resubmit.
- Read the "Allowed Imports" and "Restricted Features" to avoid losing points.
- **Check on Ed Discussion for a note containing all official clarifications and sample outputs.**

It is expected that everyone will follow the Student-Faculty Expectations document and the Student Code of Conduct. The professor expects a **positive, respectful, and engaged academic environment** inside the classroom, outside the classroom, in all electronic communications, on all file submissions, and on any document submitted throughout the duration of the course. **No inappropriate language is to be used, and any assignment deemed by the professor to contain inappropriate offensive language or threats will get a zero.** You are to use professionalism in your work. Violations of this conduct policy will be turned over to the Office of Student Integrity for misconduct.